Indian Institute of Information Technology, Bhopal (M.P.)

Department of Computer Science and Engineering

QubiFide

A BB84-Based Framework for Secure Communication and Eavesdropping Analysis

Submitted by: Jaswant Swaram (24U020113), Akshat Kumar (24U020138)

Under the guidance of: Dr. Supriyo Mandal, Assistant Professor, CSE

Academic Year: 2026 – 2027

Introduction

Every secure message depends on a secret key. This project looks at how quantum key distribution (BB84) can be used to share that key, and how the key can then protect real messages in a working software prototype.

The problem

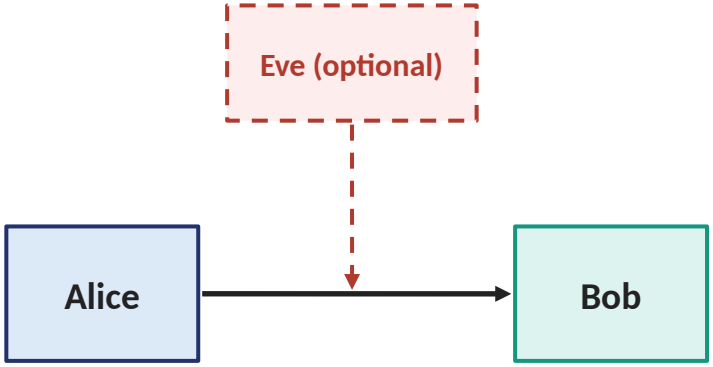
Keys must be shared safely. In classical communication, a copied key leaves no trace [1].

The quantum idea (BB84)

Bits are sent as quantum states in two bases. Measuring in the wrong basis disturbs the state, so an eavesdropper shows up as errors [1].

Our project: QubiFide

A web app that simulates BB84, optionally adds Eve, measures QBER, and uses the accepted key with HKDF + AES-GCM for encrypted chat [7], [8].



Eve's measurement disturbs the states, which shows up as errors (QBER)

BB84 simulation → QBER check → HKDF → AES-GCM → secure chat

Literature Review

Reference	Contribution	Use in our project
Bennett & Brassard [1]	Introduced BB84	Key exchange simulation
Shor & Preskill [3]	Simple security proof of BB84	Theory behind the protocol
Scarani et al. [4]	Security of practical QKD	Theory vs real systems
Lee, Sohn & Lee [5]	Detecting eavesdropping using QBER	QBER analysis
Fuchs et al. [12]	Optimal eavesdropping strategies	Effect of eavesdropping
Krawczyk & Eronen [7]	HKDF (RFC 5869)	Key derivation
Dworkin [8]	AES-GCM (NIST SP 800-38D)	Message encryption
Qiskit Aer [6]	Quantum circuit simulator	BB84 simulation

Background: BB84 (1984) showed that a key can be shared using quantum states in two bases; QKD alone does not encrypt messages. **Research gap:** these parts exist separately. We combine them (BB84 simulation, QBER, optional intercept-resend, HKDF, AES-GCM) in one web prototype. No new protocol is proposed.

Problem Statement

- Classical encryption depends on keys that have to be created and shared securely
- Interference during key exchange should be detectable before the key is used
- QBER is measured on a sample, so a low QBER does not prove nobody was listening [4], [5]
- The sifted key is not a secure key by itself; it needs checking and key derivation [4], [17]
- QKD does not encrypt messages, so classical encryption is still needed [7], [8]
- Most existing work is theoretical or hardware-based, which is hard to try out in a classroom

Why a software demo?

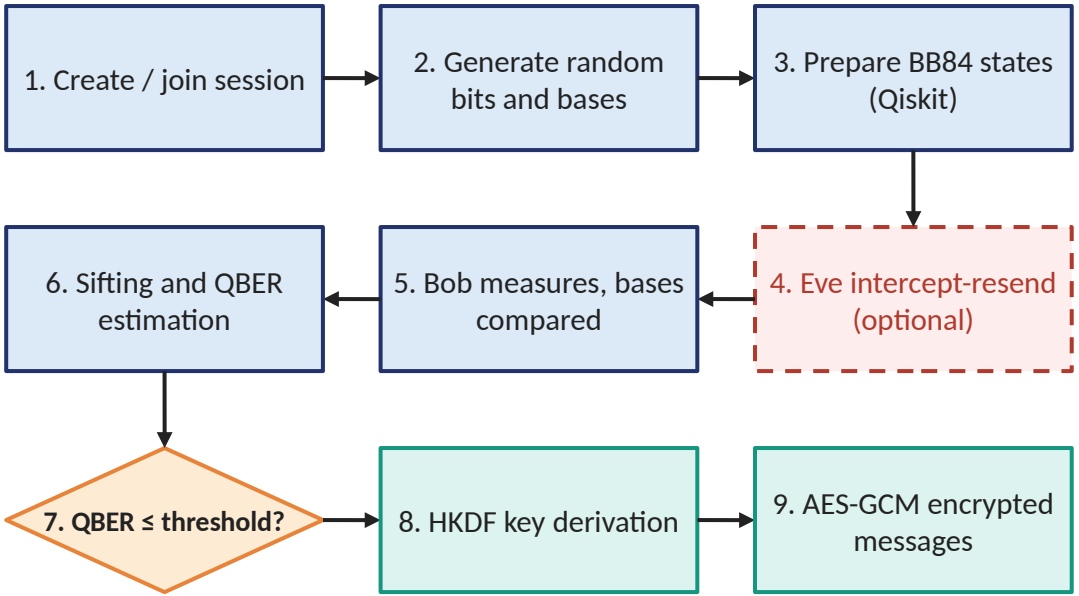
- Steps of BB84 can be seen
- Effect of Eve can be tested
- No hardware needed
- Shows how a QBER check decides if messaging is allowed

Objectives and Proposed Methodology

Objectives

1. Implement BB84 key exchange simulation
2. Demonstrate intercept-resend eavesdropping
3. Calculate and analyse QBER
4. Derive application keys using HKDF
5. Authenticated encryption using AES-GCM
6. Integrate everything in a web application

Methodology



Dashed box = optional Eve attack. If step 7 fails (QBER above threshold) the exchange is rejected and messaging stays locked.

System Diagram / Architecture

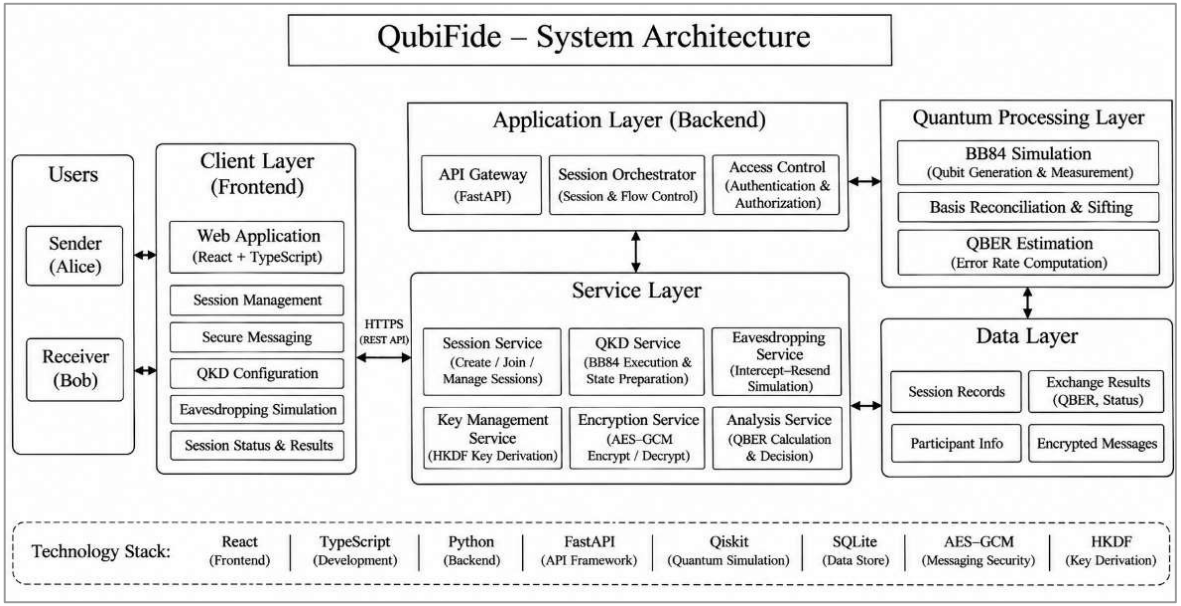


Fig. 1: System architecture

Flow: users → frontend → FastAPI backend → BB84 simulation → QBER decision → HKDF / AES-GCM. Records are stored in the data store.

- Frontend: React + TypeScript
- Backend: Python + FastAPI
- BB84 simulation: Qiskit + Qiskit Aer
- QBER and acceptance check
- HKDF + AES-GCM after acceptance
- Data store for sessions and messages

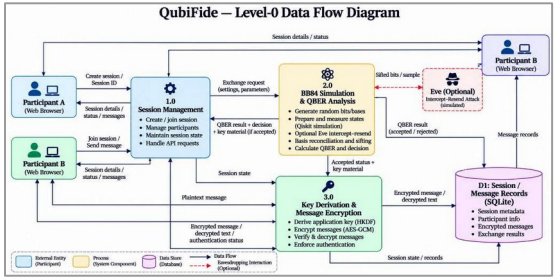


Fig. 2: Level-0 DFD

Proposed Algorithm and Flowchart

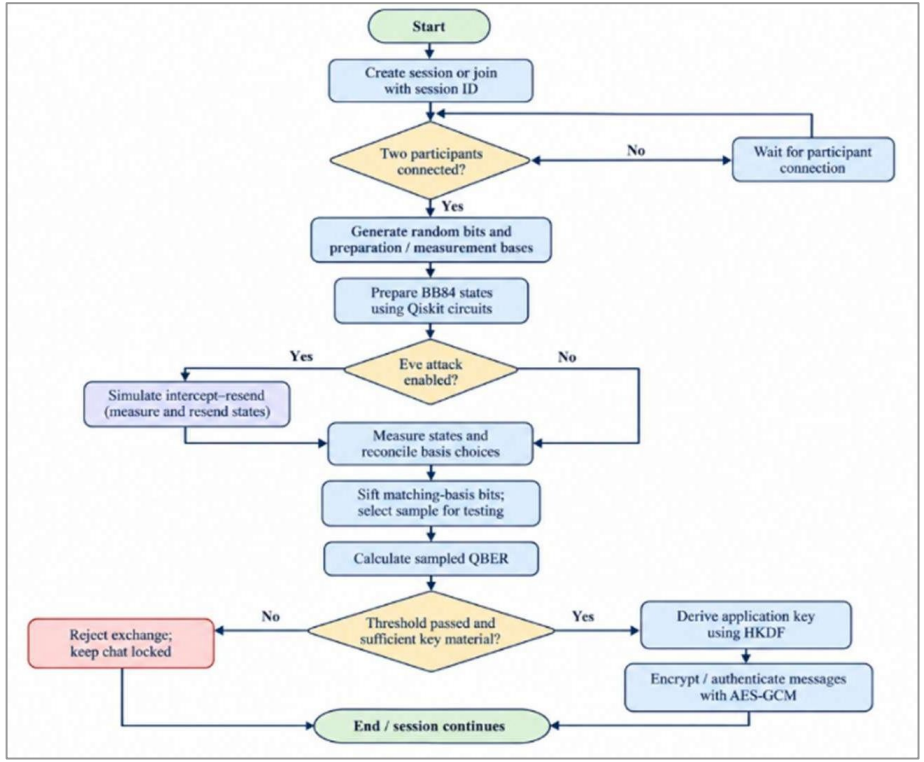


Fig. 3: Workflow of the proposed methodology

1. BB84 key distribution

Alice picks random bits and bases; Bob picks random measurement bases; matching-basis positions are kept (sifting). Expected sifted bits: E[M] = N/2

2. QBER calculation

QBER = (e / k) × 100%, where e = mismatches in k sampled bits. Reject if above threshold. Full intercept-resend: ½ × ½ = 25%

3. Key derivation and messaging

PRK = HKDF-Extract(salt, IKM)
OKM = HKDF-Expand(PRK, info, L)
OKM is the AES-GCM key; the receiver decrypts and checks the tag, otherwise the message is rejected.

Implementation

Part	Technology
Frontend	React, TypeScript, Vite
Backend	Python, FastAPI
Quantum simulation	Qiskit, Qiskit Aer
Key derivation	HKDF (SHA-256)
Encryption	AES-256-GCM
Sessions	Session ID, create / join

Messaging stays locked until the key exchange is accepted.

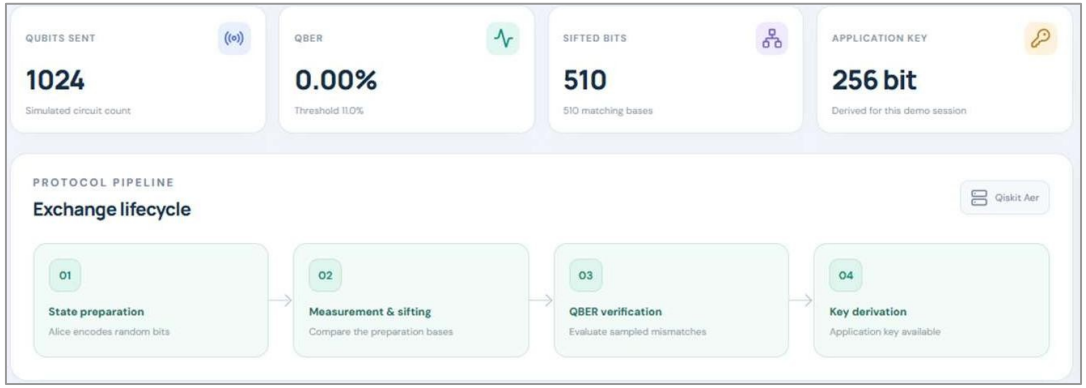
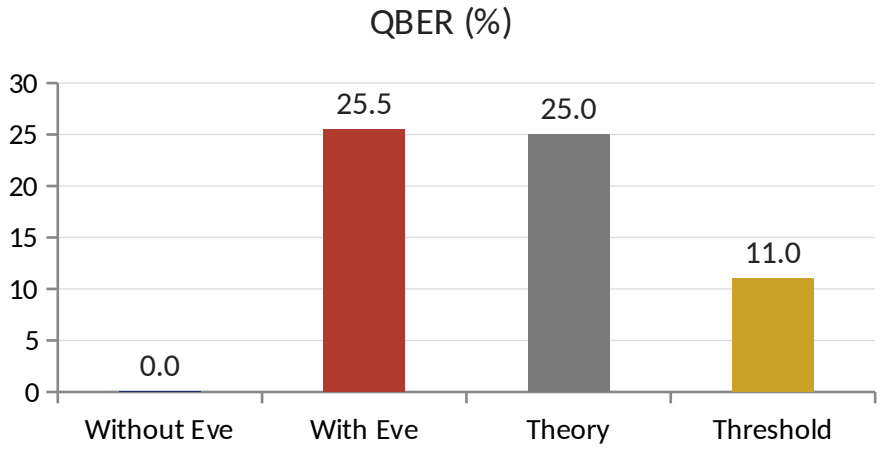


Fig. 4: Completed key exchange (QBER 0.00%, 256-bit key)



Fig. 5: Eve run. Exchange rejected: 26/102 test errors, 512 matching bases, threshold 11.0%.

Results and Analysis



- Theory: Eve on every qubit with random bases gives QBER = ½ × ½ = 25%
- Measured: 0.00% without Eve; 26/102 ≈ 25.5% with Eve (single runs, 25.5% computed from Fig. 5)
- Repeated runs still need to be added
- Low QBER does not prove there was no eavesdropper

Test	Result in report
QBER within threshold	Exchange accepted
QBER above threshold	Exchange rejected
Valid encrypt / decrypt	Message recovered
Modified ciphertext	Rejected
Modified auth tag	Authentication failure
Messaging after rejection	Stays disabled
Not enough key material	Key derivation blocked

Table: results from report Tables 2 to 4

Conclusion and Future Work

Conclusion

- BB84 key exchange simulated with Qiskit Aer
- Intercept-resend attack and QBER-based accept / reject
- Key derived with HKDF; messages protected with AES-GCM
- All integrated in a React + FastAPI web application

Future Work

- Use real quantum hardware
- Noise, loss and more eavesdropping models
- Error correction and privacy amplification
- Better key management and authentication
- More security and performance testing
- More application features (e.g. multi-party)

Limitations

- Simulation only, no real quantum hardware
- Only intercept-resend attack, no formal proof
- No error correction or privacy amplification
- Backend can access session keys

References

[1] C. H. Bennett and G. Brassard, “Quantum cryptography: Public key distribution and coin tossing,” in Proc. IEEE Int. Conf. Computers, Systems and Signal Processing, Bangalore, India, 1984, pp. 175–179.

[3] P. W. Shor and J. Preskill, “Simple proof of security of the BB84 quantum key distribution protocol,” Phys. Rev. Lett., vol. 85, no. 2, pp. 441–444, 2000.

[4] V. Scarani et al., “The security of practical quantum key distribution,” Rev. Mod. Phys., vol. 81, no. 3, pp. 1301–1350, 2009.

[5] C. Lee, I. Sohn, and W. Lee, “Eavesdropping detection in BB84 quantum key distribution protocols,” IEEE Trans. Netw. Serv. Manag., vol. 19, no. 3, pp. 2689–2701, 2022.

[6] IBM Quantum, “Qiskit Aer documentation.” [Online]. Available: https://qiskit.github.io/qiskit-aer/

[7] H. Krawczyk and P. Eronen, “HMAC-based extract-and-expand key derivation function (HKDF),” RFC 5869, IETF, May 2010.

[8] M. Dworkin, Recommendation for Block Cipher Modes of Operation: GCM and GMAC, NIST SP 800-38D, Nov. 2007.

[12] C. A. Fuchs et al., “Optimal eavesdropping in quantum cryptography. I,” Phys. Rev. A, vol. 56, no. 2, pp. 1163–1172, 1997.

[17] H.-K. Lo, M. Curty, and K. Tamaki, “Secure quantum key distribution,” Nature Photonics, vol. 8, pp. 595–604, 2014.

Acknowledgement: we thank our guide Dr. Supriyo Mandal, Dr. Rahul Chaurasia and all faculty of the CSE department, IIIT Bhopal, for their guidance and support.

Thank You